

ROS 2 Getting Started

Author: Manoj Sharma

Robot Operating System (ROS), www.ros.org, is an open source operating system designed for robots. It is “a flexible framework for writing robot software. It is a collection of tools, libraries, and conventions that aim to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.”

This guide complements the [official ROS 2 tutorial](#) . Throughout this guide, the Operating System of choice is Ubuntu (Linux) 22.04 along with Jazzy version of ROS 2 (or simply ROS). The programming language of choice is Python with a slight mix of C++.

Disclaimer

We encourage the user to use a virtual machine for running the exercises. Some of the commands listed in the guide may break the existing project environments you may have.

Contents

1	PREPARING FOR ROS	1
1.1	SETUP UBUNTU 24 (NOBLE NUMBAT).....	1
1.2	INSTALL ROS 2 (JAZZY)	1
1.3	LINUX FUNDAMENTALS	1
1.4	INTRODUCTION TO PYTHON.....	2
1.5	INTRODUCTION TO C++	2
2	ABOUT ROS 2.....	3
2.1	WHAT IS ROS.....	3
2.2	TYPES OF INTERACTION B/W ROS NODES.....	4
2.3	ROS MESSAGES.....	5
2.4	ROS 1 - ROS 2 BRIDGE [OPTIONAL].....	5
3	ROS 2 TUTORIAL - BASIC.....	6
3.1	SETUP A WORKSPACE	6
3.2	CREATE A PACKAGE	7
3.3	CREATE/ADD NODE(S) IN A PACKAGE.....	9
3.3.1	<i>PUBLISHER NODE.....</i>	<i>9</i>
3.3.2	<i>SUBSCRIBER NODE</i>	<i>9</i>
3.3.3	<i>ADD ENTRY POINTS.....</i>	<i>10</i>
3.3.4	<i>BUILD & RUN</i>	<i>10</i>
3.3.5	<i>ADDITIONAL RESOURCES/TOOLS.....</i>	<i>11</i>
3.4	LAUNCH FILE SETUP	14
3.4.1	<i>SETUP A PACKAGE (CMAKE).....</i>	<i>14</i>
3.4.2	<i>CREATE A LAUNCH FILE.....</i>	<i>15</i>
3.4.3	<i>EDIT CMAKE LIST.....</i>	<i>15</i>
3.4.4	<i>BUILD & LAUNCH</i>	<i>15</i>
3.5	EXERCISE: MODIFY ROS MESSAGE TYPE	16
4	SETUP ROS ON A SINGLE BOARD COMPUTER (SBC)	17
4.1	RASPBERRY PI	17
4.2	JETSON ORIN NANO DEVLOPER KIT	18
4.3	JETSON ORIN AGX/NX DEVLOPER KIT.....	19
5	ROS-ARDUINO/PERIPHERAL COMMUNICATION.....	20

5.1	INSTALL "PYSERIAL"	20
5.2	CLONE "ARDUINO-PACKAGE" REPOSITORY	20
5.3	UPLOAD ARDUINO EXAMPLE SKETCH	20
5.4	BUILD & RUN	21
5.5	VISUALIZE TOPICS AND INTERACT	21
6	ROS DATA VISUALIZATION AND LOGGING	22
6.1	EXAMPLE DATA PUBLISHER	22
6.1.1	CLONE REPOSITORY	22
6.1.2	BUILD & RUN	22
6.2	DATA VISUALIZATION	23
6.2.1	ROS2 TOPIC ECHO	23
6.2.2	RQT_PLOT	24
6.3	DATA LOGGING	24
6.4	EXPORTING DATA TO CSV	26
7	ROBOT MANIPULATOR CONTROL USING ROS	27
7.1	PRE-REQUISITES AND DEPENDENCIES	27
7.1.1	INSTALL "PIP"	27
7.1.2	INSTALL "pymycobot" PACKAGE	28
7.1.3	INSTALL REQUIRED DEPENDENCIES	28
7.2	SETUP W/S, CLONE THE PACKAGES, AND BUILD W/S	28
7.2.1	CLONE "mycobot_ros2" ROS-PACKAGE	28
7.2.2	CLONE "teleop_core_virtual" ROS-PACKAGE	28
7.2.3	CLONE "mycobot_interface" ROS-PACKAGE	29
7.2.4	BUILD THE WORKSPACE	29
7.3	HANDS-ON EXERCISES	29
7.3.1	VIRTUAL ROBOT CONTROL	30
8	ADDITIONAL RESOURCES	35
8.1	INSTALL MICROSOFT VISUAL CODE	35
8.1.1	X86 Install	35
8.1.2	ARM Install	35
8.2	ROS2 COMMAND CHEATSHEET	35
8.3	USB PORT HANDOVER TO VIRTUAL MACHINE	35
8.4	ISSUE - "ERROR OPENING SERIAL PORT"	35
8.5	ISSUE - ADD "sudo" RIGHTS	36

1 PREPARING FOR ROS

Preparing for ROS tutorial requires setting up Ubuntu, installing ROS, knowing the fundamentals of Linux, and a basic understanding of Python (C++ is optional).

ROS 2 Jazzy is natively compatible with Ubuntu 24. The compatibility chart is shown in Figure 1.



Figure 1: ROS 2 OS and hardware compatibility chart.

1.1 SETUP UBUNTU 24 (NOBLE NUMBAT)

Ubuntu 24 (or Ubuntu 24.04 LTS), codename Noble Numbat is a Long Term Release (LTS) with end-of-life (EOL) of 2029 is a suitable version for various project requiring Ubuntu. Below are three possible ways to set up Ubuntu 24 (pick any one of 1, 2, or 3 although, **1 is recommended**):

1. RUN UBUNTU ON A VIRTUAL MACHINE [Recommended] [Click here for instructions.](#)
2. NATIVE UBUNTU INSTALL [Click here for instructions.](#)
3. UBUNTU ON WINDOWS SUBSYSTEM FOR LINUX (WSL) [Click here for instructions.](#)

1.2 INSTALL ROS 2 (JAZZY)

Once Ubuntu 24 is setup, [click here](#) for instructions on ROS 2 Jazzy installation instructions. Additionally, a step-by-step installation guide is available [here](#).

1.3 LINUX FUNDAMENTALS

[Click here](#) to learn more about Ubuntu Graphical User Interface (GUI).

1.4 INTRODUCTION TO PYTHON

W3schools has put together a step-by-step Python tutorial that is broken into sub-components. The tutorial is [linked here](#). Be sure to complete the following components: *Intro* through *Operators*, *If...Else* and *For Loops*.

1.5 INTRODUCTION TO C++

Optional - Similarly, W3schools has put together a step-by-step C++ tutorial that is broken into sub-components. The tutorial is [linked here](#). Be sure to complete the following components: *Intro* through *Operators*, *If...Else* and *For Loops*.

2 ABOUT ROS 2

ROS 2 is a ground-up reimagining of ROS 1¹. ROS 2 was started in 2014. As of 2023 ROS 2 is on eighth named release, called *Jazzy*. ROS 2 Jazzy uses Ubuntu 22 as a host OS to run. Its [End-of-Life \(EoL\)](#) date is May 2027.

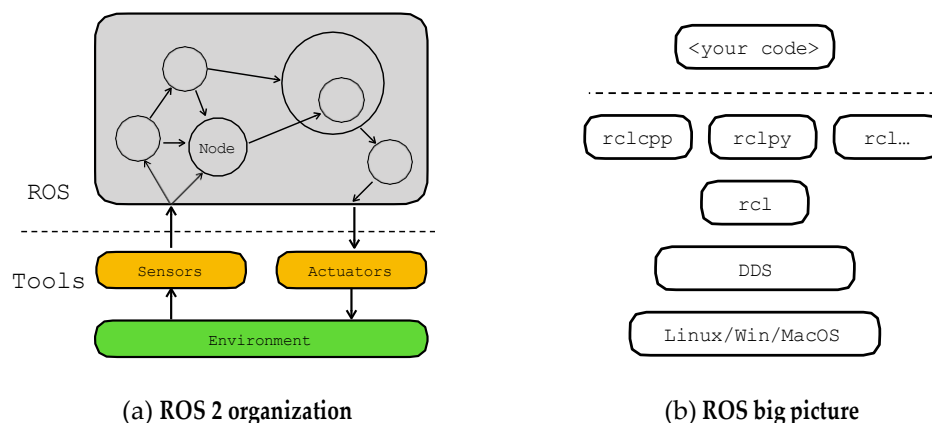


Figure 2: ROS 2 architecture.

ROS 2 is a middleware designed to run on Linux, Windows or MacOS. It is built on DDS², a middleware proven in industry. Figure 2b shows this structure. ROS client library is used to communicate between the nodes.

2.1 WHAT IS ROS

ROS allows a problem to be broken down into smaller pieces, [nodes](#) and [packages](#) which could be developed as a separate entity and can be combined together (similar to *Lego* pieces). It provides a framework, tools, and interfaces for distributed development. In essence, ROS is:

- **PLUMBING** Allows data/information between nodes as shown in Figure 2a.
- **TOOLS** Logging, plotting, visualization, diagnostics. [Tools cheat sheet](#)
- **CAPABILITIES** Planning, perception, execution.
- **ECOSYSTEM** Encourages re-use of software pieces and share among the community.

¹ROS 1 has been around since 2008. It uses custom TCP/IP middleware for the communication network. Its latest release, *Noetic*, has EOL date in May 2025.

²[Click here](#) for more details on ROS 2 design.

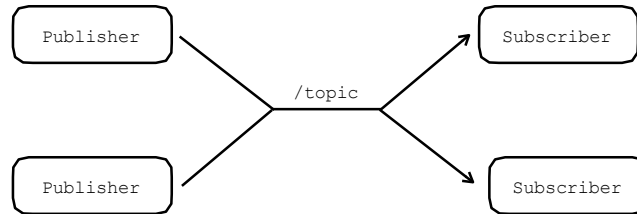


Figure 3: Message: Publisher - Subscriber model.

2.2 TYPES OF INTERACTION B/W ROS NODES

ROS offers three core types of interactions between ROS nodes. *Messages* are the simplest type where a publisher continuously broadcasts the message at a user defined rate on a *topic*, refer to Figure 3, (similar to a radio station broadcasts on a specific frequency) to which a subscriber can acquire data by subscribing (similar to tuning a radio to a specific frequency). *Service* and *Actions* are asynchronous means of interact between nodes. Table 1 summarizes the strengths and weaknesses of the three core interaction types.

Type	Strengths	Weaknesses
<i>Message</i>	Good for most sensors One-to-many	Easy to overload Message drop w/o knowledge
<i>Service</i>	Knowledge of missed call Well defined feedback	Blocks until completion Connection typically re-establishes for each service call (slow activity)
<i>Actions</i>	Monitor long running processes Handshake	Complicated

Table 1: Three core types of interaction between ROS nodes.

2.3 ROS MESSAGES

ROS uses a [simplified messages](#) description language for describing the data values (or messages) that ROS nodes publish and/or subscribe:

1. Standard messages, [std_msgs](#), represent primitive data types and other basic message constructs, such as multiarrays.
2. Sensor messages, [sensor_msgs](#), represent messages for commonly used sensors, including cameras and scanning laser rangefinders.
3. Geometry messages, [geometry_msgs](#), represent geometric primitives such as points, vectors, and poses.
4. Other messages include [action_msgs](#), [diagnostic_msgs](#), [nav_msgs](#), [shape_msgs](#), [stereo_msgs](#), [trajectory_msgs](#), [visualization_msgs](#), and [<custom_msgs>](#).
5. **Industrial-core messages, [industrial_core](#), are the messages unique to the industrial manipulator widely used across the industry. Click [here](#) for more details on ROS Industrial.**

2.4 ROS 1 - ROS 2 BRIDGE [OPTIONAL]

There are several instances where a ROS 1 workspace can't be easily converted or ported over to ROS 2 and therefore unable to communicate with ROS 2 workspaces. A workaround to that is to use a ROS 1 to ROS 2 bridge which, as the name suggests, bridges the communication. Click [here](#) for more details.

3 ROS 2 TUTORIAL - BASIC

This section covers four main topics - setup a workspace (refer to Figure 4), create a package, create a simple publisher & subscriber nodes, and lastly a launch file to run multiple nodes using a simple launch file.

For additional details, refer to the official tutorial on ROS 2- [click here](#).

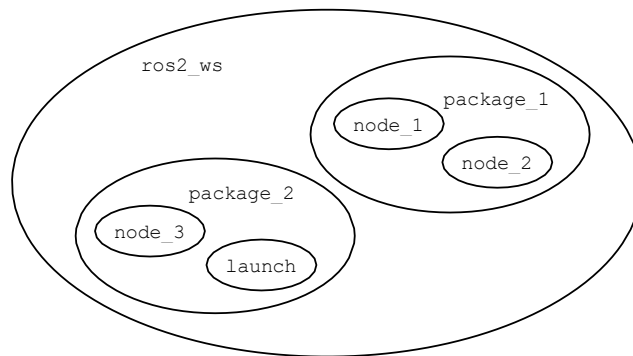


Figure 4: A typical ROS workspace architecture.

3.1 SETUP A WORKSPACE

Setting up a workspace refers to creating a workspace directory of choice, resolving the package dependencies (if any), followed by building the workspace.

The first step is to source the ROS 2 environment:

```
source /opt/ros/jazzy/setup.bash
```

Create a workspace in the */home* directory by creating a parent directory of a custom name and a *src* directory within it. Here is how to create a workspace named *ros2_ws*:

```
cd ~/
mkdir -p ~/ros2_ws/src/
```

The directory *ros2_ws* is also known as root directory of this workspace. To move to the root directory from *home* directory:

```
cd ~/ros2_ws/
```

Resolve any package dependencies:

```
rosdep install -i --from-path src --rosdistro jazzy -y
```

Finally, build the workspace:

```
colcon build
```

FYI - The console will return the following (no need to run this commands):

```
Summary: 0 package finished [0.16 s]
```

3.2 CREATE A PACKAGE

A package may consist of logical programs (or nodes) for a specific task. This package example consists of a simple subscriber and a publisher.

Navigate to your *src* directory:

```
cd ~/ros2_ws/src
```

Create a package with a name of your choice using *ament python* tools. Syntax to create a package named '*my_package*':

```
ros2 pkg create --build-type ament_python my_package
```

Next step is to build the package. To do so, move back to the root workspace directory (note - you must be in the root workspace to build³):

```
cd ~/ros2_ws/
```

Build the package:

³unless any specific packages are installed.

```
colcon build
```

FYI - The console will return the following (no need to run these commands):

```
Starting >>> my_package
```

```
Finished <<< my_package [1.71 s]
```

```
Summary: 1 package finished [1.80 s]
```

Note - if console responds with "colcon not found" then try installing colcon:

```
sudo apt install python3-colcon-common-extensions
```

3.3 CREATE/ADD NODE(S) IN A PACKAGE

A package may consist of logical programs (or nodes) for a specific task. This example consists of a simple subscriber and a publisher.

Nodes are programs designed for a specific task. A node may consist of subscriber(s) and/or publisher(s). To add a node, navigate to the following directory:

```
cd ~/ros2_ws/src/my_package/my_package/
```

3.3.1 PUBLISHER NODE

A sample publisher python script is available on GitHub.

- [click here](#). Download/duplicate this python script in the directory mentioned above.

To keep it simple, use the same file name, i.e. *publisher_member_function.py*. This script initializes the node as *minimal_publisher*, starts to publish 'Hello World' followed by a timestamp on a topic aptly named */topic*, and prints it on the console at every 0.5s. Be sure to read the documentation to understand more about this code.

3.3.2 SUBSCRIBER NODE

A sample subscriber python script is available on GitHub.

- [click here](#). Download/duplicate this python script in the directory mentioned above.

To keep it simple, use the same file name, i.e. *subscriber_member_function.py*. This script initializes the node as *minimal_subscriber*, subscribes to a topic aptly named */topic*, and prints it on the console as it receives the message. Be sure to read the documentation to understand more about this code.

FYI - At this point there should be two additional nodes (one publisher and one subscriber) present within *my_package*. To see these files run:

```
ls ~/ros2_ws/src/my_package/my_package/
```

The console will return the following (no need to run these commands):

```
__init__.py publisher_member_function.py
```

`subscriber_member_function.py`

3.3.3 ADD ENTRY POINTS

This is done by editing *setup.py*, located in:

```
/ros2_ws/src/my_package/
```

Create two entry points for *publisher_member_function.py* and *subscriber_member_function.py* named *pub* and *sub*, respectively. To do so, open *setup.py* using an editor of your choice, scroll to the bottom of the script, and add the following within '*console_scripts*':

```
'pub = my_package.publisher_member_function:main',  
'sub = my_package.subscriber_member_function:main',
```

FYI - The completed *setup.py* file is also available on GitHub - [click here](#).

3.3.4 BUILD & RUN

Navigate to the root of the workspace and then build the package:

```
cd ~/ros2_ws/  
colcon build
```

Note - every time changes are made in package and/or nodes, e.g. Python script(s), be sure to "build" and source the workspace before running the node.

To run the publisher - open a new terminal, navigate to the root of the workspace, source ROS 2 environment, and source the workspace:

```
cd ~/ros2_ws/  
source /opt/ros/jazzy/setup.bash  
source install/setup.bash
```

Use *ros2 run* command to run *pub* node which is present in *my_package* package:

```
ros2 run my_package pub
```

The console will start publishing info, something like this:

```
[INFO] [minimal_publisher]: Publishing: "Hello World: 0"
```

```
[INFO] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [minimal_publisher]: Publishing: "Hello World: 2"
...
```

To run the subscriber - open a new terminal, navigate to the root of the workspace, source ROS 2 environment, and source the workspace:

```
cd ~/ros2_ws/
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

Use `ros2 run` command to run `pub` node which is present in `my_package` package:

```
ros2 run my_package sub
```

The console will start publishing info, something like this:

```
[INFO] [minimal_subscriber]: I heard: "Hello World: 10"
[INFO] [minimal_subscriber]: I heard: "Hello World: 11"
[INFO] [minimal_subscriber]: I heard: "Hello World: 12"
```

3.3.5 ADDITIONAL RESOURCES/TOOLS

ROS 2 has several tools often helpful in de-bugging. A few of them are listed below. For more information, refer to a ROS 2 command guide - [click here](#).

While the two nodes are running (in two separate terminals), open another terminal, source the ROS 2 environment:

```
source /opt/ros/jazzy/setup.bash
```

A GUI tool called `rqt_graph` provides a pictorial representation of all the active nodes and ROS structure, shown in Figure 5. To run `rqt_graph` run:

```
ros2 run rqt_graph rqt_graph
```

This will open up a new window showing the active ROS 2 architecture. This can also be used to verify that all the nodes and active properly connected.

Similarly, there are many more useful tools for e.g., *ros2 topic list*, *ros2 topic echo <topic name>*, *ros2 topic info <topic name>*, etc. ...

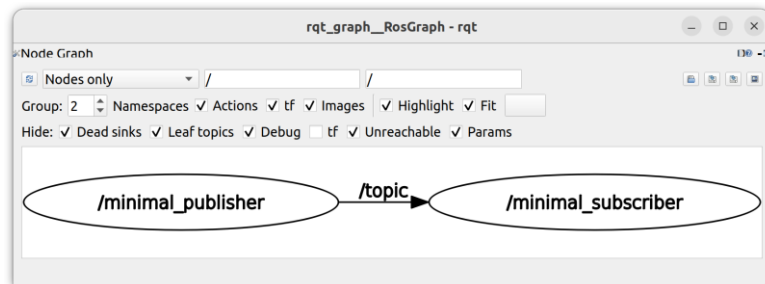


Figure 5: Graphical representation of active nodes and topics.

Note - to stop any activity on a terminal press *Ctrl* + *C*.

3.4 LAUNCH FILE SETUP

A typical workspace may contain several packages where each package may contain multiple nodes in it. This means running multiple nodes one-by-one is very time consuming. A launch file is designed to run and/or launch several nodes and/or launch files. One way to architect this is to create a new package and containing the launch file(s).

3.4.1 SETUP A PACKAGE (CMAKE)

In the case of a package consisting of launch file(s) it is preferred to build a package using *ament cmake* tools (and not *ament python*). To create a package, navigate to your *src* directory:

```
cd ~/ros2_ws/src
```

Syntax to create a package named '*my_launch_pkg*':

```
ros2 pkg create --build-type ament_cmake my_launch_pkg
```

Build the package. To do so, move back to the root workspace directory:

```
cd ~/ros2_ws/  
colcon build
```

FYI - The console will return the following (no need to run these commands):

```
Starting >>> my_package  
Starting >>> my_launch_pkg  
Finished <<< my_package [1.27 s]  
Finished <<< my_launch_pkg [1.57 s]
```

```
Summary: 2 package finished [1.75 s]
```

Next, create a *launch* directory in */ros2_ws/src/my_launch_pkg/*, and change directory to the newly created *launch* directory:

```
cd ~/ros2_ws/src/my_launch_pkg  
mkdir launch  
cd launch
```

3.4.2 CREATE A LAUNCH FILE

A sample file python launch script is available on GitHub - [click here](#). Download/duplicate this python script inside the *launch* directory with a *my_first.launch.py* name. Note that the *entry points* for *pub* & *pub* (Section 3.3.3) acts as executable which are referenced in *my_first.launch.py*. This means launching a (single) *launch* file will run the publisher and subscriber node.

3.4.3 EDIT CMAKE LIST

Lastly the *CMakeLists.txt* in */ros2_ws/src/my_launch_pkg/* needs to be updated by adding the following:

```
install (DIRECTORY launch DESTINATION share/${PROJECT_NAME})
```

Refer to an updated *CMakeLists.txt* on GitHub - [click here](#).

3.4.4 BUILD & LAUNCH

Navigate to the root of the workspace and then build the package:

```
cd ~/ros2_ws/  
colcon build
```

To launch the *my_first.launch.py* file (in the *my_launch_pkg* package) source the workspace and use the launch command:

```
cd ~/ros2_ws/  
source install/setup.bash  
ros2 launch my_launch_pkg my_first.launch.py
```

If done correctly, the console should output the publisher and subscriber info similar to the ones discussed in Section 3.3.4.

Additionally, both the nodes can be verified by running *rqt_graph* which is discussed in Section 3.3.5.

Note that, here, (by design) one launch file is able to start two nodes. It is good practice to run all the logical nodes via one launch file. There may be cases where a single node needs to be run one at a time for the purpose of debugging.

[Optional] To avoid sourcing the ROS2 environment every time you open a new shell, then you can add the command to your shell startup script

```
echo "source /opt/ros/jazzy/setup.bash" » ~/.bashrc
```

Note - sometimes the computer may have multiple versions of ROS installed and therefore the user may need to decide which version to source. In such scenario the above step is not recommended as it may cause confusion.

3.5 EXERCISE: MODIFY ROS MESSAGE TYPE

At this point, you have created a simple publisher and subscriber nodes. The publisher node is publishing "Hello World" message of *String* type over a topic called */topic*. Complete the following exercise:

1. Modify the publisher node to publish today's date of *Int16* type over a topic named */date*.
(Hint: Remember that after making changes to python scripts, e.g. nodes, be sure to "build" and source the workspace before running the node.)
2. Use the CLI tools (for e.g. "ros2 topic echo ...") to verify listen to the */date* topic.
3. Open "rqt_graph" to verify that the publisher and subscriber are communicating with each other. Hint - refer to Figure 5.

4 SETUP ROS ON A SINGLE BOARD COMPUTER (SBC)

Often times a control system needs to be deployed on a target SBC such as [Raspberry Pi 4](#), [Raspberry Pi 5 \(Model B\)](#)⁴, and [NVIDIA Jetson](#).

4.1 RASPBERRY PI

Items needed - Raspberry Pi model B, USB-C power supply, micro-SD card⁵, mini-HDMI to HDMI cable, keyboard & mouse, and the internet connection.

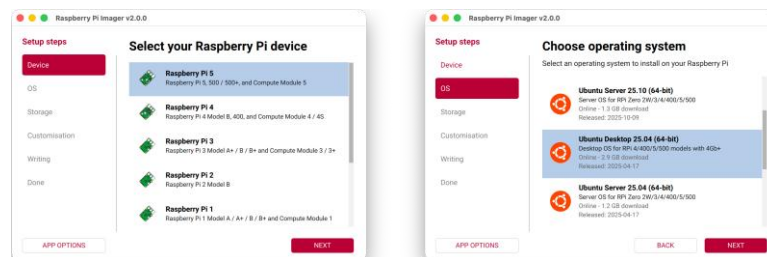


Figure 6: Raspberry Pi Imager: From *Device* → select *Raspberry Pi 5*; *OS* → select *Other gen-purpose OS* → *Ubuntu* → *Ubuntu Desktop 24.04.03 LTS (64-bit)*. Select appropriate *Storage*, skip *Customization* and click *Write*. The newer version may have a different layout, but the general process should be similar.

1. **Insert micro-SD card** - Insert the card into the computer; you may need an adapter if the computer doesn't have a slot.
2. **Prepare micro-SD card** - [Raspberry Pi Imager](#) is an official tool available to create a bootable card for Raspberry pi.
3. **Write the Image** - Open [Raspberry Pi Imager](#), Select [Raspberry Pi 5](#) as the target device then select [Ubuntu 24.04.3 LTS](#), and click "NEXT" (shown in Figure 6).
4. **Set up the OS** - Connect the Raspberry pi to a display, power, and follow the screen prompts to set up the OS. Note - using generic password like *ubuntu* helps to conveniently access the account over the network.
5. **Install ROS** - install [ROS 2 jazzy](#).
6. **Install SSH Server** - [optional] *Secure Shell (SSH)* allows the Raspberry pi to be controlled via SSH via a different computer. [Open SSH Server installation..](#)

⁴preferably 4GB RAM or more.

⁵preferably 32GB or more.

4.2 JETSON ORIN NANO DEVELOPER KIT

Items needed - Raspberry Pi model B, USB-C power supply, micro-SD card⁶, mini-HDMI to HDMI cable, keyboard & mouse, and the internet connection.

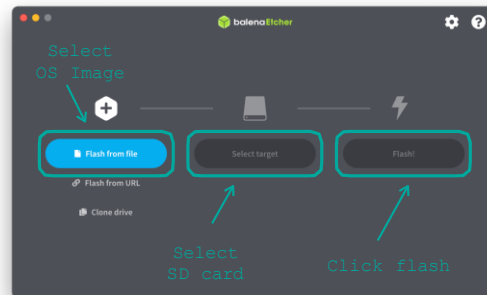


Figure 7: Balena Etcher. Select *Flash from file*, select *target* and then click *Flash!*

1. **Prepare micro-SD card** - *Balena Etcher* is a tool available to create a bootable card for Jetson Orin Nano.
2. **Download Jetpack** - The (Ubuntu) OS designed by NVIDIA for Jetson Orin Nano is called *Jetpack*. Click here to download *Jetpack 6.0*, which is based on Ubuntu 22.04 LTS.
3. **Write the Image** - Launch *Balena Etcher*, select *Jetpack 6.0*, and then follow the prompts shown in Figure 6 to write the image.
4. **Set up the OS** - Connect the Jetson to a display, power, and follow the screen prompts to set up the OS. Note - using generic password like *ubuntu* helps to conveniently access the account over the network.
5. **Install ROS** - install *ROS 2 Humble*⁷.

⁶preferably 32GB or more.

⁷Note that the *Jetpack 6.x* is based on Ubuntu 22, and therefore ROS2 Humble should be installed.

4.3 JETSON ORIN AGX/NX DEVELOPER KIT

Other family of Jetson Orin, for eg. *Jetson Orin AGX* or *Jetson Orin NX*, require *NVIDIA SDK Manager* to install the OS (or Jetpack). Once done, install *ROS 2 Foxy*.

5 ROS-ARDUINO/PERIPHERAL COMMUNICATION

A simple serial-communication protocol is an easy way to establish ROS-ARDUINO communication. In this tutorial, we use an Arduino to broadcast two integer comma-separated values ending with a newline. Simultaneously, the Arduino can receive an 8-bit integer value and convert it into an LED brightness. A sample script is [available here](#).

5.1 INSTALL “PYSERIAL”

Install *pyserial* module:

```
sudo apt update
sudo apt install pipx
pipx install pyserial
```

Use: `sudo apt install python3-pip` instead of `pipx`
Use: `sudo apt install python3-serial` instead of `pipx`

5.2 CLONE “ARDUINO-PACKAGE” REPOSITORY

Navigate to your *src* directory and clone the repository (which is [arduino_pkg](#)):

```
cd ~/ros2_ws/src
git clone https://github.com/irahulone/arduino_pkg.git
```

5.3 UPLOAD ARDUINO EXAMPLE SKETCH

An example Arduino-sketch is available under the *src* directory of *arduino_pkg*; alternatively, [click here](#) to download the sketch. Upload the sketch to Arduino.

Note - When using a Virtual Machine (VM), the Arduino Mega may not be recognized. To hand over the USB-COM port from the host (Windows) to Ubuntu, navigate to *Device* → *USB* → and select Arduino Mega. For more information, refer to Section [8.2](#).

[Optional:] Use the following command to install Arduino IDE, if you don't have it. Arduino IDE is needed to upload the Arduino-script.

```
sudo apt install arduino
```

5.4 BUILD & RUN

Navigate to the root of the workspace, build the package, and run the node:
(If the console responds with “*Error opening serial port,*” refer to Section 8.3.)
(Select the appropriate serial port for the Arduino; default is /dev/ttyACM0/)

Build the package:

```
cd ~/ros2_ws/  
source /opt/ros/jazzy/setup.bash  
colcon build
```

Run the package:

```
source /opt/ros/jazzy/setup.bash  
source install/setup.bash  
ros2 run arduino_pkg read_write
```

5.5 VISUALIZE TOPICS AND INTERACT

In a new terminal check the available topics (value_x, value_y) and *echo* them to verify the data reception.

Use the following command to publish the value(s) on a topic (write_z) directly from the console ([click here](#) for more details):

```
ros2 topic pub write_z std_msgs/Int16 "data: 40"
```

We know that the number 40 is the value sent to the Arduino, which is then translated into the brightness of the onboard LED. Try replacing this number with a different one ranging from 0 to 255 and observe the brightness of the LED. change corresponding to the number.

6 ROS DATA VISUALIZATION AND LOGGING

To learn more about visualization, logging, and exporting ROS data, let's use a dummy package called, `fake_data_broadcaster` [available on GitHub \(click here\)](#). This package contains an example node; upon starting, it starts to broadcast a modeled temperature, humidity, and time information on a typical day⁸ on topics

`\temperature`, `\humidity`, and a few more, respectively.

6.1 EXAMPLE DATA PUBLISHER

6.1.1 CLONE REPOSITORY

Navigate to `src` directory and clone the `fake_data_broadcaster` repository :

```
cd ~/ros2_ws/src
git clone https://github.com/irahulone/fake_data_broadcaster.git
```

6.1.2 BUILD & RUN

Navigate to the root of the workspace and then build the package:

```
cd ~/ros2_ws/
colcon build
```

Note - every time changes are made in package and/or nodes, e.g. Python script(s), be sure to "build" and source the workspace before running the node.

Source the environment / workspace:

```
cd ~/ros2_ws/
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

Use `ros2 run` command to run `fake_weather_broadcaster`:

```
ros2 run weather_data_publisher weather_data_publisher_node
```

Once this node is running, the temperature, humidity, and time data will begin to broadcast on `\temperature`, `\humidity`, and a few more topics, respectively.

⁸The data are published at 1 Hz, with each successive value corresponding to a 15-minute increment. This means that the 24-hour day is represented by 96 discrete data points, spanning 98 seconds. The node continues to broadcast in a loop until it is stopped.

6.2 DATA VISUALIZATION

Data, in ROS environment, can be visualized in various ways. Tools such as *ros2 topic echo* and *rqt_plot* can be used to visualize a data in numeral-format and plot-format, respectively.

Note - various third party tools exists (for e.g., [PlotJuggler](#)) with advanced plotting visualization and plotting capabilities.

6.2.1 ROS2 TOPIC ECHO

Use this command to list all the available topics:

```
ros2 topic list
```

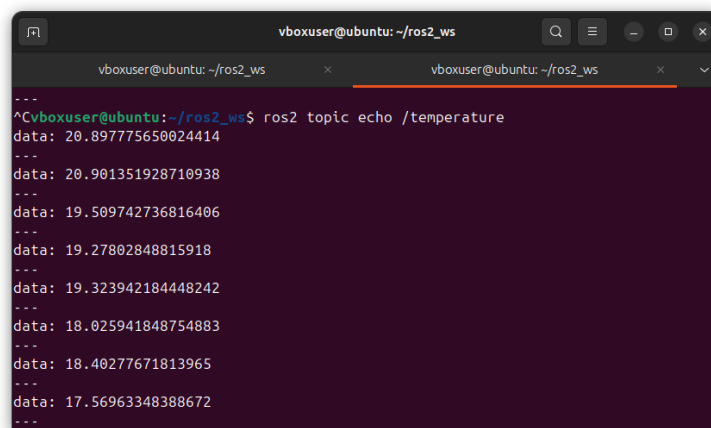
To see the data being published on a topic in a human readable, here is the general syntax (no need to run this command):

```
ros2 topic echo <topic_name>
```

For example, to see the data being published on a topic named `/temperature` is:

```
ros2 topic echo /temperature
```

You should see the data in real time in the terminal, as shown in Figure 8.



```
vboxuser@ubuntu: ~/ros2_ws
^Cvboxuser@ubuntu:~/ros2_ws$ ros2 topic echo /temperature
data: 20.897775650024414
data: 20.901351928710938
data: 19.509742736816406
data: 19.27802848815918
data: 19.323942184448242
data: 18.025941848754883
data: 18.40277671813965
data: 17.56963348388672
```

Figure 8: Example of a *ros2 topic echo* window.

6.2.2 RQT_PLOT

It is a tool that provides a simple and quick way to visualize the flow of numerical data from a topic in real time. It's perfect for debugging, monitoring, and understanding the data flow.

To launch *rqt_plot* window, run this command in a new terminal window:

```
ros2 run rqt_plot rqt_plot
```

This should launch an *rqt_plot* window (you may have to resize the window), shown in Figure 9.

One open, add the topic by typing the topic name followed by */data* in the dialog box in the top left corner of the window (e.g. */humidity/data*).

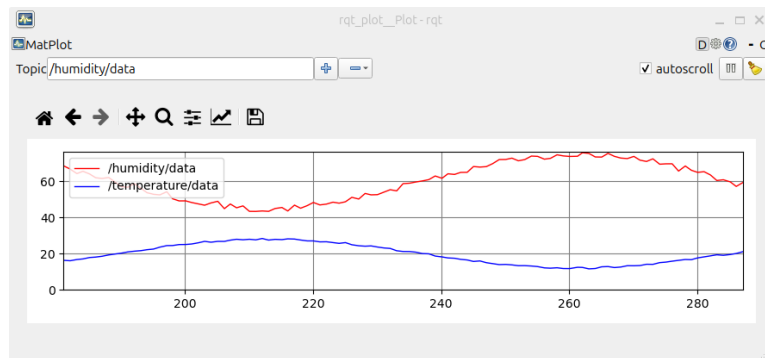


Figure 9: Example of an *rqt_plot* window.

6.3 DATA LOGGING

ROS has an inbuilt tool, called *rosvag record* ([click here for official documentation](#)), to record the timestamped messages from various topics, services and actions during runtime. It is a part of the *rosvag* toolset, which is used to record, play back, and analyze ROS message data. When you use *rosvag record*, it saves the messages published on the selected topics to a file, which can later be used for analysis, debugging, or reproducing experiments.

Let's create a directory (folder) named *my_rosvag_data* in the home directory and move into this newly created directory. In a new terminal:

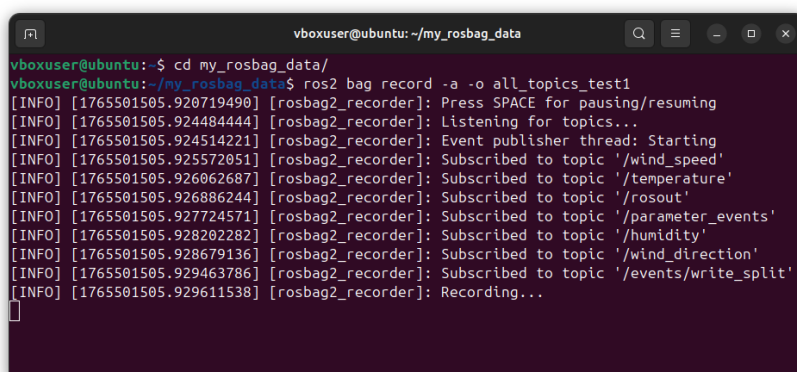
Note - feel free to use a custom name and location of your choice.

```
cd ~/
mkdir my_rosbag_data
cd my_rosbag_data
```

While the ros publisher node is running (refer to Sec. 6.1.2), run the following command to initiate the recording process into a custom bag file named “all_topics_test1”, where “-a” flag is shorthand for recording all active topics:

```
ros2 bag record -a -o all_topics_test1
```

The terminal should show a response that confirms all the topics that are being recorded, as shown in Figure 10. Make sure that the topics of importance are cross-checked in the response.



```
vboxuser@ubuntu: ~/my_rosbag_data
vboxuser@ubuntu:~/my_rosbag_data$ cd my_rosbag_data/
vboxuser@ubuntu:~/my_rosbag_data$ ros2 bag record -a -o all_topics_test1
[INFO] [1765501505.920719490] [rosbag2_recorder]: Press SPACE for pausing/resuming
[INFO] [1765501505.924484444] [rosbag2_recorder]: Listening for topics...
[INFO] [1765501505.924514221] [rosbag2_recorder]: Event publisher thread: Starting
[INFO] [1765501505.925572051] [rosbag2_recorder]: Subscribed to topic '/wind_speed'
[INFO] [1765501505.926062687] [rosbag2_recorder]: Subscribed to topic '/temperature'
[INFO] [1765501505.926886244] [rosbag2_recorder]: Subscribed to topic '/rosout'
[INFO] [1765501505.927724571] [rosbag2_recorder]: Subscribed to topic '/parameter_events'
[INFO] [1765501505.928202282] [rosbag2_recorder]: Subscribed to topic '/humidity'
[INFO] [1765501505.928679136] [rosbag2_recorder]: Subscribed to topic '/wind_direction'
[INFO] [1765501505.929463786] [rosbag2_recorder]: Subscribed to topic '/events/write_split'
[INFO] [1765501505.929611538] [rosbag2_recorder]: Recording...
```

Figure 10: rosbag response window.

Once done, use keyboard command *ctrl* + *c* to terminate the recording process.

Note - for this exercise, let it record for about 10-20 s.

6.4 EXPORTING DATA TO CSV

There are various ways to export the recorded data/topics into comma-separated values (.csv) file. Let's go through a simple example where we export the /wind_speed topic into a file named *wind_speed.csv* using a Python script.

Note - before continuing, make sure you have recorded data for approximately 10-20 s, as described in Sec. 6.3. (The exact time is not crucial; what matters is the presence of data, as the example script will be looking for a file to operate on.)

Step 1: Use file browser to locate the *bag_to_csv.py* Python script under the ".../utils/" directory within the *fake_data_broadcaster*.

(Alternatively, [click here](#) to download *bag_to_csv.py*.)

Step 2: Use the file browser to copy (or move) *bag_to_csv.py* to *.../my_rosbag_data/all_topics_test1/* directory.

Step 3: Open a new terminal window and navigate to *.../my_rosbag_data/all_topics_test1/* directory by running the following command:

```
cd ~/my_rosbag_data/all_topics_test1/
```

In the same terminal, run *bag_to_csv.py*:

```
python3 bag_to_csv.py
```

Once complete, you should see a *wind_speed.csv* created in the working directory. Open the .csv file to examine the two columns of data: first one is the times-stamp (the computer's internal time), followed by the data published on the topic /wind_speed.

[Optional] - feel free to modify *bag_to_csv.py* script to export a different topic of your choice.

Note - The .csv file can be used to plot the data in tools such as MATLAB, MS Excel, etc.

7 ROBOT MANIPULATOR CONTROL USING ROS

A robot manipulator, including the industrial manipulators such as Universal Robot (UR), Emika Franka Panda, etc., can be controlled either via their propriety control software or ROS. Learn more about ROS-based industrial robot control [here](#).

ROS offers various manipulator-specific tools essential to design control such as – [MoveIt](#) and [RViz](#). These tools exist in addition to the regular ROS tools (e.g. `rqt_graph`, `ros2 topic list`, etc.) that are covered in the previous chapters.

To learn more about the manipulator specific tools, let's use a [MyCobot 280](#) from Elephant Robotics which is a 6-DoF robot manipulator that closely resembles conventional industrial manipulator. In this chapter we will develop the control “pipeline” using pre- designed and configured programs and/or components.

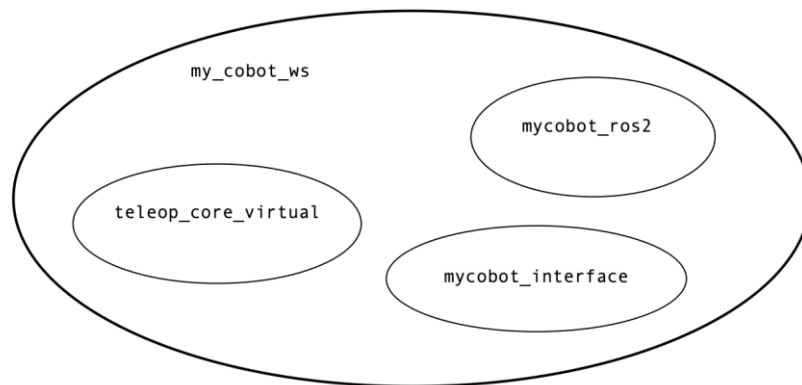


Figure 11: *my_cobot_ws* (workspace) structure consisting of three packages.

Prior to setting up the workspace, let's install the prerequisites and setup the required dependencies -

7.1 PRE-REQUISITES AND DEPENDENCIES

7.1.1 INSTALL “PIP”

If you haven't already installed pip, refer to [Section 5.1](#) for install instructions.

7.1.2 INSTALL “pymycobot” PACKAGE

```
python3 -m pipx install pymycobot
```

Use : pip3 install pymycobot (if there is a permission issue include --break-system-packages at the end)

7.1.3 INSTALL REQUIRED DEPENDENCIES

Need to change from jazzy version to humble for all following commands if jetson is used. Use pyqt5 instead of pyqt6 for jetson

```
sudo apt install python3-pyqt6
sudo apt install ros-jazzy-xacro
sudo apt install ros-jazzy-joint-state-publisher-gui
sudo apt install ros-jazzy-moveit
sudo apt install ros-jazzy-tf-transformations
```

7.2 SETUP W/S, CLONE THE PACKAGES, AND BUILD W/S

Let's create a workspace (w/s) called “*mycobot_ws*”.

```
cd ~/
mkdir -p ~/mycobot_ws/src/
```

For this exercise, we need the following three packages in the workspace
Note, remember that you should clone the packages in the ‘src’ directory in your workspace.

7.2.1 CLONE “mycobot_ros2” ROS-PACKAGE

Clone *mycobot_ros2* package – [link](#) to instructions on GitHub. This is package offered and maintained by the robot manufacturer containing the libraries needed to communicate with the robot.

7.2.2 CLONE “teleop_core_virtual” ROS-PACKAGE

Clone *teleop_core_virtual* package – [link](#) to instructions on GitHub. This package is offered and maintained by RSL that provides and virtual interface to simulate robot inputs in the form of interactive sliders.

7.2.3 CLONE “mycobot_interface” ROS-PACKAGE

Clone *mycobot_interface* package – [link](#) to instructions on GitHub. This package is also offered and maintained by RSL that provides and virtual interface to simulate robot inputs in the form of interactive sliders.

7.2.4 BUILD THE WORKSPACE

Important – for convenience, “symlink” to build the workspace. This would enable us to make changes to the nodes (script) and then run them without re-building the workspace. To build the workspace (remember, you need to be in the root w/s directory before building) -

Use humble instead of jazzy if jetson is used

```
cd ~/mycobot_ws/  
source /opt/ros/jazzy/setup.bash  
colcon build --symlink-install
```

FYI – due to robot model and additional robot specific features, this build may take a few minutes to complete the build process.

7.3 HANDS-ON EXERCISES

Be sure to refer to the robot kinematic to understand the robot-specify frame assignments – [link to the product documentation page](#).

The hands-on exercises cover three distinct modes of robot control –

1. Interactive: a simple and interactive with all the necessary nodes are presented alongside the guide.
2. Joint-space: user need to create a “ros node” to publish the *desired joint angels* on a topic to control the robot.
3. Cartesian-space: user need to create a “ros node” to publish the *desired cartesian coordinates* on a topic to an “IK-solver” node that solves for the corresponding joint angels and feeds them to the robot. (IK-solver is presented alongside the guide.)

The above three modes could be executed in “virtual robot” and/or “real robot”. The following sections contains the step-by-step procedure for the virtual robot.

Troubleshoot if there will be an error when using colcon build. In Jetson orin nano ubuntu 22 version might not install it automatically. colcon needs to be primarily install via sudo apt (system wise).

* check if colcon is installed -> 'which colcon' -> output: /usr/bin/colcon

* If not presented -> 'sudo apt install python3-colcon-common-extensions'

7.3.1 VIRTUAL ROBOT CONTROL

Let's walk through one exercise at a time:
What you need – your computer only

Exercise 1 – Interactive robot control

[Three-step process]

As the name suggests, we rely on a program where interactive sliders are used to control a robot.

Step 1: First, launch an “interactive slider program” (this is developed and maintained by RSL):

```
cd ~/mycobot_ws/
source install/setup.bash
ros2 launch teleop_core_virtual virtual_controller.launch.py
```

in `virtual_controller.py` file (not the launch file) change all PyQt6 to PyQt5 at the beginning when importing libraries. This applies to jetson. Ubuntu 22 use old python wheel packages. Therefore PyQt6 is not compatible.

Step 2: In a second terminal, launch the “robot model alongside RViz” (MyCobot package is developed and maintained by the robot manufacturer):

```
cd ~/mycobot_ws/
source install/setup.bash
ros2 launch mycobot_interface joystick_control_vir.launch.py
```

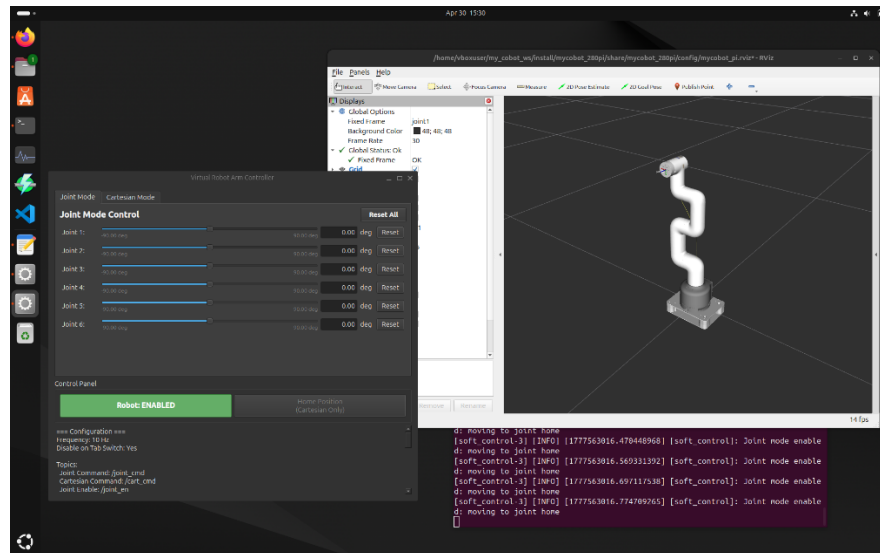


Figure 12: Example desktop depicting “virtual controller” and “RViz”.

At this point, two program windows should appear on the desktop as shown in Fig 12. One is the virtual joystick and the other is RViz window with the robot model.

Step 3: Using the “virtual controller”: on top of the window, there are two tabs – Joint Mode and Cartesian mode. Near the bottom of the window you will find “Robot Enable” button. It turns green when enabled. Click “enable” and explore the robot motion in Joint as well as Cartesian modes by moving the sliders!

Exercise 2 – Joint space control

[Four-step process]

The objective of this exercise is to **create a node** that publishes desired joint angles on a topic named “/joint_cmd” of message type “[Float64MultiArray](#)”. The message should consist of six elements that corresponds to six joint angles in degrees. To avoid self-collision, pick the angles within +/- 30 degrees.

Notes

1. Feel free to create a node withing “*mycobot_interface*” package (refer to [Section 7.2.3](#)) and name the node (which is a python script) as “*des_jstate_publisher.py*”. For simplicity, use the same name i.e. “*des_jstate_publisher*” as the entry point.
2. If unsure, feel free to this combination of des joint angels: [10, 30, -30, 20, 5, 0] in your publisher node.

Step 1: Create the node and add the entry point, it’s time to try send the desired joint angels to the virtual robot in RViz.

Step 2: Build the workspace (refer to [Section 7.2.4](#)).

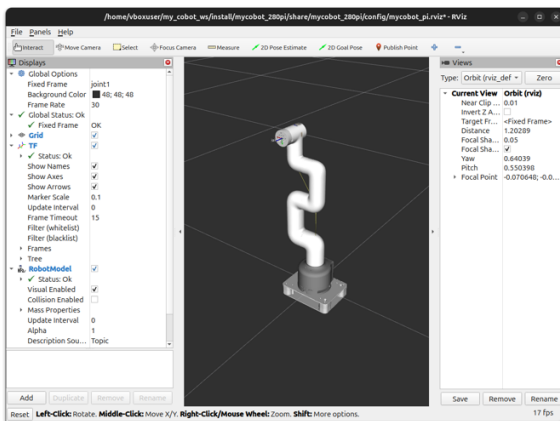
When rebuilding after changes to the coordinates, use 'colcon build --packages-select mycobot_interface' to save time

Step 3: Launch the “virtual robot”. This will allow a virtual robot to appear within RViz (Fig. 13.a) and activate robot controller such that the (virtual) robot is ready to accept (subscribe) joint commands:

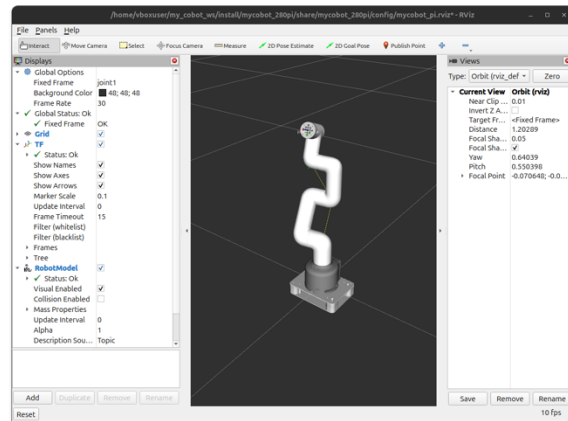
```
cd ~/mycobot_ws/  
source install/setup.bash  
ros2 launch mycobot_interface mycobot_280pi_vir.launch.py
```

Step 4: In a new terminal, run the “*des_jstate_publisher*”. This will publish the desired joint angels to the robot controller:

```
cd ~/mycobot_ws/
source install/setup.bash
ros2 run mycobot_interface goal_joint_state_pub
```



(a) Step 3



(b) Step 4

Figure 13: Joint mode: example desktop depicting robot state in “RViz”.

If the execution is successful, then you should see the robot moving from its home position to the new position that reflect the desired joint angels as shown in Fig. 13.b.

Exercise 3 – Cartesian space control

[Five-step process]

The objective of this exercise is to **create a node** that publishes desired end-effector (EE) position and orientation (in robot's base frame relative to a home position) on a topic named `"/cart_cmd"` of message type `"Twist"`.

For simplicity, begin with a desired position (and not the orientation) by setting x , y and z values (which is software limited to ± 100 mm for x , y , and z). The message should consist of three elements that corresponds to x , y and z in mm.

Notes

1. Feel free to create a node withing `"mycobot_interface"` package (refer to [Section 7.2.3](#)) and name the node (which is a python script) as `"des_cstate_publisher.py"`. For simplicity, use the same name i.e. `"des_cstate_publisher"` as the entry point.
2. If unsure, feel free to this combination of x , y and z : `[-30, 20, 45]` in your publisher node. You may set the orientation about x , y and z as `[0, 0, 0]`.

Step 1: Create the node and add the entry point, it's time to try send the desired robot EE position to the virtual robot in RViz.

Step 2: Build the workspace (refer to [Section 7.2.4](#)).

When rebuilding after changes to the coordinates, use `'colcon build --packages-select mycobot_interface'` to save time

Step 3: Launch the "virtual robot". This will allow a virtual robot to appear within RViz (Fig. 14.a) and activate robot controller such that the (virtual) robot is ready to accept (subscribe) joint commands:

```
cd ~/mycobot_ws/  
source install/setup.bash  
ros2 launch mycobot_interface mycobot_280pi_vir.launch.py
```

Step 4: In a new terminal, run an "IK-solver" to ensure that that the incoming cartesian commands can be converted to joint commands and sent over to the robot controller:

```
cd ~/mycobot_ws/  
source install/setup.bash  
ros2 run mycobot_interface ik_solver
```

Step 5: Run the “*des_cstate_publisher*”. This will publish the desired EE position to the robot controller:

```
cd ~/mycobot_ws/
source install/setup.bash
ros2 run mycobot_interface goal_cart_state_pub
```

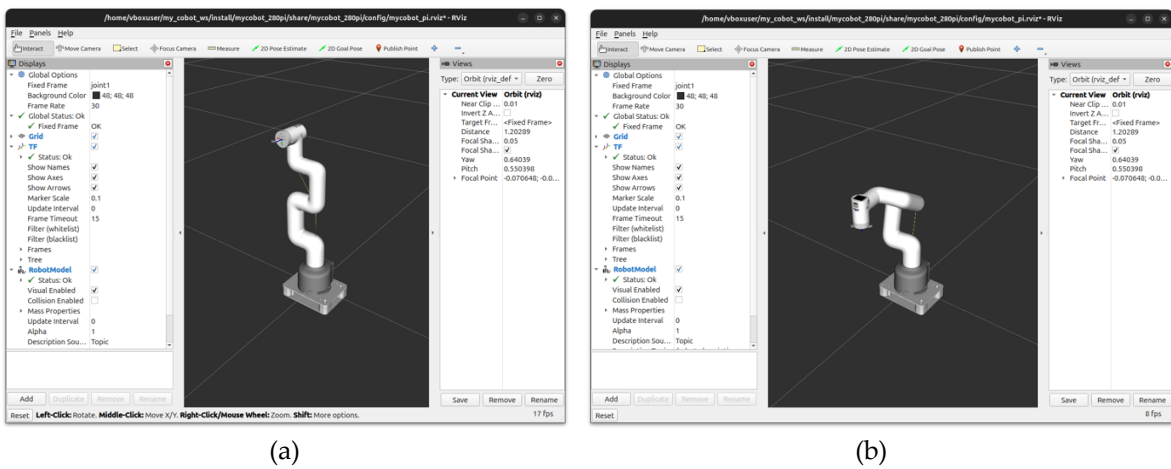


Figure 14: Cartesian mode: example desktop depicting robot state in “RViz”.

If the execution is successful, then you should see the robot moving from its home position to the new position that reflect the desired new position as shown in Fig. 14.b.

8 ADDITIONAL RESOURCES

8.1 INSTALL MICROSOFT VISUAL CODE

8.1.1 X86 Install

For Ubuntu running on x86, open Ubuntu Software, search for *visual studio code*. Generally, the first result app is the VS Code, click *install* and follow the directions.

8.1.2 ARM Install

For Ubuntu running on ARM, for example Raspberry pi, NVIDIA Jet-sons, or other ARM architecture-based devices:

```
sudo apt update
sudo apt install code
```

8.2 ROS2 COMMAND CHEATSHEET

Here is a list of commonly used ROS 2 commands [ROS 2 tools cheat sheet](#).

8.3 USB PORT HANDOVER TO VIRTUAL MACHINE

Arduino may require additional steps when connected to Virtual Machine (VM) to ensure the serial port is recognized by the VM.

Refer to this how to guide on [Control Arduino from Ubuntu VirtualBox](#) by The Robotics Back-End.

8.4 ISSUE - "ERROR OPENING SERIAL PORT"

Accessing serial port requires additional permission. To set serial port permission, first determine the serial port:

```
ls /dev/tty*
```

The serial port may be USBx or ACMx. If the port in question is USBx, run the following command

```
ls -l /dev/ttyUSB
```

FYI - The console may return something similar

```
crw-rw---- 1 root dialout 188, 0 5 apr 23.01 ttyUSB0
```

Add the Linux user to the dialout group (replace the <username> with the appropriate username)

```
sudo usermod -a -G dialout <username>
```

Log out and log in again (or reboot) for this change to take effect.

8.5 ISSUE - ADD "*sudo*" RIGHTS

[link here](#)